```
            }
            System.out.println();
        }
```

με χρήση της switch.

6 Πίνακες

Οι πίνακες είναι συλλογές δεδομένων ίδιου τύπου. Για παράδειγμα, ένας πίνακας ακέραιων αποθηκεύει μια σειρά από τιμές τύπου int ή ένας πίνακας πραγματικών αποθηκεύει μια σειρά από τιμές τύπου double. Όπως εξηγούμε αναλυτικά παρακάτω σε αυτήν την ενότητα, η προσπέλαση στα στοιχεία ενός πίνακα είναι η ταχύτερη δυνατή σε σχέση με την προσπέλαση στα στοιχεία συλλογής οποιουδήποτε άλλου τύπου.

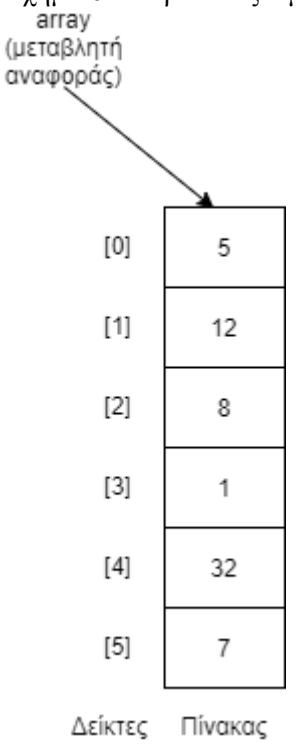
Στο πλαίσιο της Java, ένας πίνακας τα στοιχεία του οποίου δεν είναι πίνακες ονομάζεται μονοδιάστατος πίνακας. Ένας πίνακας τα στοιχεία του οποίου είναι μονοδιάστατοι πίνακες ονομάζεται δισδιάστατος πίνακας. Αντίστοιχα, ένας πίνακας τα στοιχεία του οποίου είναι δισδιάστατοι πίνακες ονομάζεται τρισδιάστατος πίνακας, κόκ. Συχνά οι μη μονοδιάστατοι πίνακες αναφέρονται ως πολυδιάστατοι.

Σε αυτό το κεφάλαιο, παρουσιάζουμε τις συντακτικές λεπτομέρειες ορισμού των μονοδιάστατων και πολυδιάστατων πινάκων, εξηγούμε βασικά στοιχεία της ενσωματωμένης στην Java, κλάσης Arrays που παρέχει μια σειρά από συναρτήσεις για τη διαχείριση πινάκων και παρουσιάζουμε μια σειρά από βασικές επεξεργασίες που αφορούν τους πίνακες.

6.1 Μονοδιάστατοι Πίνακες

Για τη διαχείριση ενός πίνακα, χρειάζεται στο πρόγραμμά μας να έχουμε τουλάχιστον μια μεταβλητή μέσω της οποίας μπορούμε να προσπελάσουμε τον πίνακα και τα στοιχεία του. Στην Java, οι μεταβλητές μέσα από τις οποίες προσπελαύνουμε πίνακες είναι μεταβλητές αναφοράς και όχι τιμής, δηλαδή αποθηκεύουν τη διεύθυνση μνήμης στην οποία είναι κατανεμημένος ο πίνακας. Επιπλέον, τα στοιχεία ενός μονοδιάστατου πίνακα κατανέμονται σε διαδοχικές θέσεις στη μνήμη. Σε αυτό το χαρακτηριστικό οφείλεται η ταχύτατη προσπέλαση των στοιχείων ενός πίνακα.

Στο σχήμα 6.1 παρουσιάζουμε έναν πίνακα ακέραιων.



Σχήμα 6.1 *Αναπαράσταση μονοδιάστατου πίνακα ακέραιων*

Ο πίνακας του σχήματος 6.1 έχει μήκος 6 και περιέχει τους ακέραιους: 5, 12, 8, 1, 32, 7. Η μεταβλητή αναφοράς array περιέχει τη διεύθυνση της αρχής του πίνακα. Οι θέσεις του πίνακα είναι αριθμημένες. Στη θέση 0, για παράδειγμα, είναι τοποθετημένη η τιμή 5, στη θέση 1 είναι τοποθετημένη η τιμή 12, κοκ.

Για να προσπελάσουμε ένα στοιχείο του πίνακα χρειαζόμαστε τρία στοιχεία. Τη μεταβλητή αναφοράς που δείχνει στην αρχή του πίνακα, έναν δείκτη (index) που δείχνει τη θέση μέσα στον πίνακα του στοιχείου που θέλουμε να προσπελάσουμε και τον τελεστή θέσης (index operator) που συμβολίζεται ως []. Σύμφωνα με τα παραπάνω, η έκφραση array[2] προσπελαύνει τη θέση με δείκτη 2 του πίνακα array όπου είναι αποθηκευμένη η ακέραια τιμή 8. Αντίστοιχα, array[3]==1, array[4]==32, κλπ.

Οι δείκτες των πινάκων στην Java αριθμούνται πάντα αρχίζοντας από το μηδέν (zero based). Έτσι, ένας πίνακας με μήκος ν έχει δείκτες από 0 έως ν-1. Ο πίνακας του σχήματος 6.1 για παράδειγμα, έχει μήκος 6 και δείκτες από 0 έως 5. Επομένως, αν επιχειρήσουμε να προσπελάσουμε το στοιχείο array[6] επιχειρούμε στην ουσία να προσπελάσουμε περιοχή έξω από τη μνήμη που έχει δεσμευτεί για τον πίνακα. Τέτοια ενέργεια συνιστά λάθος χρόνου εκτέλεσης και παράγει κατάλληλη εξαίρεση.

Όπως αναφέραμε στην αρχή αυτής της ενότητας, οι πίνακες είναι οι ταχύτερες συλλογές. Πιο συγκεκριμένα, οι πίνακες είναι δομές τυχαίας προσπέλασης (random access), δηλαδή δομές στις οποίες ο χρόνος που απαιτείται για την προσπέλαση οποιουδήποτε στοιχείου είναι σταθερός. Με άλλα λόγια, χρειαζόμαστε τον ίδιο χρόνο για να προσπελάσουμε ένα στοιχείο στη θέση 1000 ενός πίνακα με τον χρόνο που απαιτεί η προσπέλαση στη θέση 0 του πίνακα.

Στον πίνακα του σχήματος 6.1, το στοιχείο στη θέση 0 βρίσκεται στη διεύθυνση μνήμης array+0*Integer.SIZE και το στοιχείο στη θέση 1 βρίσκεται στη διεύθυνση μνήμης array+1*Integer.SIZE. Γενικότερα, το στοιχείο στη θέση k ενός μονοδιάστατου πίνακα ακέραιων βρίσκεται στη διεύθυνση αρχής του πίνακα + k*Integer.SIZE. Επομένως, για να υπολογιστεί η διεύθυνση ενός οποιουδήποτε στοιχείου του πίνακα, χρειαζόμαστε να υπολογίσουμε μόνο μια απλή έκφραση.

Αυτή η ευκολία είναι αποτέλεσμα της διαδοχικής κατανομής στη μνήμη των στοιχείων των μονοδιάστατων πινάκων. Το μειονέκτημα βέβαια της διαδοχικής κατανομής είναι πως μετά τη δημιουργία δεν είναι δυνατόν να μεταβληθεί το μέγεθος του πίνακα. Ωστόσο, αυτό το μειονέκτημα, εύκολα μπορούμε να το παρακάμψουμε όπως θα δούμε παρακάτω στην άσκηση 6.5.7.

6.1.1 Δημιουργία μονοδιάστατων πινάκων

Σε αυτήν την ενότητα παρουσιάζονται οι τρόποι βασικής διαχείρισης των μονοδιάστατων πινάκων. Πιο συγκεκριμένα, θα δούμε πώς δηλώνουμε έναν μονοδιάστατο πίνακα, πώς τον αρχικοποιούμε και πώς προσπελαύνουμε τα στοιχεία του.
Στον κώδικα 6.1, δηλώνουμε, δημιουργούμε και αρχικοποιούμε τα στοιχεία ενός μονοδιάστατου πίνακα ακέραιων.

```
1   int[] array;
2   array = new int[3];
3   array[0] = 1;
4   array[1] = 3;
5   array[2] = 5;
```

Κώδικας 6.1 *Δήλωση, δημιουργία και αρχικοποίηση μονοδιάστατου πίνακα ακέραιων*

Στη γραμμή 1 του κώδικα 6.1 δηλώνουμε τη μεταβλητή array που ο τύπος της είναι πίνακας ακέραιων. Σε αυτήν τη φάση δεν έχει δημιουργηθεί ακόμη ο πίνακας. Έχει απλά δηλωθεί μια μεταβλητή κατάλληλη για τη διαχείριση μονοδιάστατων πινάκων ακέραιων. Αν θέλαμε μονοδιάστατους πίνακες πραγματικών αριθμών, θα έπρεπε να δηλώσουμε double[] array. Γενικότερα, μπορούμε να δηλώσουμε πίνακες οποιουδήποτε τύπου στην Java.

Στη γραμμή 2, δημιουργείται ο πίνακας. Η δημιουργία του επιτυγχάνεται με τη χρήση της λέξης-κλειδί new που συνοδεύεται από τον αριθμό 3 μέσα σε τετράγωνες παρενθέσεις (square brackets). Επομένως, η εντολή new int[3], δεσμεύει στη μνήμη τρεις διαδοχικές θέσεις που στην κάθε μία μπορεί να αποθηκευτεί ένας ακέραιος. Επιπλέον, στη συγκεκριμένη περίπτωση η new επιστρέφει μεταβλητή τύπου

int[], την οποία εκχωρούμε στη μεταβλητή array που δηλώσαμε στη γραμμή 1. Στις γραμμές 3 έως 5, τοποθετούμε στις θέσεις 0, 1 και 2 τις τιμές 1, 3, 5, αντίστοιχα.
Στη συνέχεια, μπορούμε να προσπελάσουμε ένα προς ένα τα στοιχεία του πίνακα χρησιμοποιώντας μια απλή for, όπως δείχνει ο κώδικας 6.2.

```
        for (int i = 0; i < array.length; i++) {
            System.out.println("Η τιμή στην θέση " + i + " είναι " +
array[i]);
        }
```

Κώδικας 6.2 *Εμφάνιση των στοιχείων μονοδιάστατου πίνακα ακέραιων*

Όπως φαίνεται στον κώδικα 6.2, προσπελαύνουμε τα στοιχεία του πίνακα στις θέσεις 0 έως array.length-1. Αυτό καταρχάς σημαίνει ότι η δομή τύπου int[] που επιστρέφει η new περιλαμβάνει εκτός από τη διεύθυνση μνήμης στην οποία κατανεμήθηκε ο πίνακας και το μήκος του πίνακα. Παρότι ο συγκεκριμένος πίνακας είναι μήκους 3, συνίσταται η χρήση της length, καθώς όπως θα δούμε στην ενότητα 7 είναι δυνατόν ένας πίνακας να περάσει παραμετρικά σε έναν κώδικα και η χρήση σταθεράς να μας οδηγήσει σε προβλήματα.
Αντίστοιχα με τους άλλους τύπους, έτσι και για τους πίνακες, δήλωση και δημιουργία μπορούν να γίνουν στην ίδια γραμμή όπως δείχνει ο κώδικας:

```
int[] array=new int[3];
```

Επίσης, παρέχεται η δυνατότητα, δήλωση, δημιουργία και αρχικοποίηση να πραγματοποιηθούν στην ίδια γραμμή, όπως φαίνεται στον κώδικα που ακολουθεί:

```
int[] array=new int[]{1,3,5,7};
```

Τέλος, δήλωση, δημιουργία και αρχικοποίηση να πραγματοποιηθούν στην ίδια γραμμή με τη βοήθεια ενός αρχικοποιητή (initializer). Ο κώδικας που ακολουθεί δημιουργεί έναν πίνακα πραγματικών με στοιχεία 2.5, 4.0, 6.8.

```
double[] table={2.5, 4.0, 6.8};
```

Στο παράδειγμα αυτό ο αρχικοποιητής είναι η παράσταση {2.5, 4.0, 6.8}.

6.1.2 Ενισχυμένη for

Όπως αναφέραμε στην ενότητα 5.2.3 περί του βρόχου for, υποστηρίζεται μια διευρυμένη μορφή της for που σχετίζεται με συλλογές όπως οι πίνακες.
Η γενική μορφή της ενισχυμένης (enhanced) for έχει ως εξής:

```
for (dataType item : collection) (
     statement(s)
}
```

Επομένως, στην ενισχυμένη for, ορίζουμε καταρχάς μια μεταβλητή, τοποθετούμε άνω και κάτω τελεία και στη συνέχεια μια μεταβλητή που αντιπροσωπεύει μια συλλογή, π.χ. έναν πίνακα. Για παράδειγμα:

```
for (double currentItem : table) {
     System.out.println(currentItem+" ");
}
```

Τον κώδικα αυτόν μπορούμε να τον διαβάσουμε ως εξής: Για κάθε ένα στοιχείο του πίνακα table που είναι τύπου double και ονομάζουμε currentItem εκτέλεσε τις προτάσεις μέσα στο μπλοκ που ακολουθεί. Σε κάθε επαναληπτικό βήμα, μέσα στο μπλοκ έχουμε πρόσβαση στο τρέχον στοιχείο του table. Εκείνο που πρέπει να προσέξουμε είναι ότι η πρόσβαση που έχουμε στα στοιχεία του table με αυτήν την for είναι μόνο για διάβασμα (read only). Με άλλα λόγια, η μεταβλητή currentItem είναι τοπική μεταβλητή της for. Σε κάθε επαναληπτικό βήμα, αντιγράφεται η τιμή ενός στοιχείου του table στην currentItem. Επομένως, μέσα στο μπλοκ δεν προσπελαύνουμε άμεσα το στοιχείο του πίνακα, αλλά μόνο ένα αντίγραφό του.
Με τη βοήθεια των παραδειγμάτων που ακολουθούν διευκρινίζουμε αυτό το ζήτημα.

```
static void alterArray() { //Κώδικας 6.3
    int[] array = {1, 3, 5};
    for (int i = 0; i < array.length; i++) {
        System.out.print(array[i] + " ");
        array[i] += 1;
    }
    for (int i = 0; i < array.length; i++) {
        System.out.println(array[i] + " ");
    }
}
```

Κώδικας 6.3 *Μεταβολή των στοιχείων του array*

Στον κώδικα 6.3 αρχικοποιούμε καταρχάς το array. Στη συνέχεια, σε κάθε επαναληπτικό βήμα μιας κλασικής for, εμφανίζουμε ένα στοιχείο του array και αμέσως μετά προσαυξάνουμε το στοιχείο κατά 1. Έξω από την πρώτη επαναληπτική διαδικασία αναπτύσσεται μια δεύτερη που σκοπό έχει να εμφανίσει τα στοιχεία του array. Πράγματι την πρώτη φορά, εμφανίζονται οι αρχικές τιμές 1, 3, 5 και στη δεύτερη for οι τιμές αυξημένες κατά 1, δηλαδή οι τιμές 2, 4, 6.

```
static void noAlteration() { //Κώδικας 6.4
    int[] array = {1, 3, 5};
    for (int cElement : array) {
        System.out.print(cElement + " ");
        cElement += 1;
    }
    System.out.println();
    for (int i = 0; i < array.length; i++) {
        System.out.print(array[i] + " ");
    }
    System.out.println();
}
```

Κώδικας 6.4 *Μεταβολή της τοπικής μεταβλητής cElement*

Αντίθετα, ο κώδικας 6.4, εμφανίζει κατά την πρώτη for τις τιμές 1, 3, 5 αλλά εμφανίζει τις ίδιες τιμές και κατά τη δεύτερη for. Αυτό συμβαίνει γιατί όπως εξηγήσαμε, η cElement δεν είναι παρά ένα αντίγραφο του τρέχοντος στοιχείου του array. Επομένως, η προσαύξηση κατά 1 που κάνουμε μέσα στην πρώτη for αφορά το αντίγραφο και όχι το στοιχείο του array. Όπως φαίνεται από την έξοδο του κώδικα, τα στοιχεία του πίνακα παραμένουν αμετάβλητα.

6.1.3 Δημιουργία αντιγράφων

Θα συζητήσουμε τώρα πώς μπορούμε να παράγουμε αντίγραφα ενός πίνακα ή ενός τμήματος πίνακα. Καταρχάς θα πρέπει να αποσαφηνίσουμε τις επιπτώσεις που προκύπτουν από το γεγονός ότι οι μεταβλητές με τις οποίες διαχειριζόμαστε τους πίνακες είναι μεταβλητές αναφοράς.
Ας δούμε λίγο τον κώδικα 6.5:

```
1     int[] array = {1, 3, 5};
2     int[] table;
3     table = array;
4     array[1] = 9;
5     System.out.println(table[1]);
```

Κώδικας 6.5 *Εκχώρηση μεταξύ πινάκων*

Στον κώδικα 6.5 δημιουργούμε καταρχάς τον πίνακα ακέραιων array. Στη συνέχεια δηλώνουμε έναν πίνακα επίσης ακέραιων που ονομάζουμε table. Στην ουσία, η table είναι μια μεταβλητή αναφοράς τύπου int[]. Στο επόμενο βήμα εκχωρούμε την array στην table. Εφόσον η array είναι μία δομή που διατηρεί τη διεύθυνση και το μήκος του πίνακα που δημιουργήθηκε με την new, αυτές είναι οι πληροφορίες που αντιγράφονται στην table. Επομένως, η table δείχνει στον ίδιο πίνακα με αυτόν που δείχνει η array. Με άλλα λόγια αυτή η εκχώρηση δεν δημιουργεί αντίγραφο του πίνακα αλλά απλώς αντίγραφο της μεταβλητής διαχείρισης του πίνακα. Έτσι, όταν στη γραμμή 4 μεταβάλλουμε σε 9 την τιμή του στοιχείου στη θέση 1 του array και στη συνέχεια εμφανίζουμε το στοιχείο στη θέση 1 του table, βλέπουμε στην οθόνη μας να εμφανίζεται το 9. Είναι αποτέλεσμα του γεγονότος ότι και οι δύο μεταβλητές αναφέρονται στον ίδιο πίνακα στη μνήμη.
Επομένως, η προσέγγιση του κώδικα 6.5 δεν είναι κατάλληλη για τη δημιουργία ενός αντιγράφου πίνακα. Ωστόσο είναι εύκολο να κάνουμε έναν κώδικα που δημιουργεί αντίγραφο ενός πίνακα.

```
int[] array = {1, 3, 5};
int[] table = new int[array.length];
for (int i = 0; i < array.length; i++) {
    table[i] = array[i];
}
array[1] = 9;
System.out.println(table[1]);
```

Κώδικας 6.6 *Δημιουργία αντιγράφου μονοδιάστατου πίνακα*

Στον κώδικα 6.6 δημιουργούμε καταρχάς τον μονοδιάστατο πίνακα ακέραιων, array. Αμέσως μετά, δημιουργούμε επίσης έναν μονοδιάστατο πίνακα ακέραιων που ονομάζουμε table και έχει μήκος ίσο με το μήκος του array. Στη συνέχεια, με τη βοήθεια μιας for αντιγράφουμε ένα προς ένα τα στοιχεία του array στον table. Οι τελευταίες 2 γραμμές επιδεικνύουν ότι μεταβολές των στοιχείων του array δεν επηρεάζουν τα στοιχεία του table.

Η μέθοδος του κώδικα 6.6 για τη δημιουργία πινάκων μας δίνει μεν σωστό αντίγραφο, έχει όμως ένα μειονέκτημα. Για τη δημιουργία του αντιγράφου χρειάζεται να προσπελάσουμε, ένα προς ένα, όλα τα στοιχεία των δύο πινάκων. Αυτό στην περίπτωση που έχουμε πίνακες με μεγάλο μήκος στο πρόγραμμά μας μπορεί να θεωρηθεί χρονοβόρο. Με δεδομένο ότι τα στοιχεία ενός πίνακα είναι κατανεμημένα σε διαδοχικές θέσεις της μνήμης, θα ήταν πολύ ταχύτερο αν μπορούσαμε να αντιγράψουμε όλο το τμήμα της μνήμης στο οποίο είναι κατανεμημένος ο αρχικός πίνακας. Η Java μας παρέχει δύο συναρτήσεις με τις οποίες μπορούμε να παράγουμε αντίγραφα πινάκων ή τμημάτων πινάκων χωρίς να χρειάζεται να προσπελάσουμε ένα προς ένα τα επιμέρους στοιχεία. Πρόκειται για την arraycopy στατική συνάρτηση της κλάσης System και την copyOfRange στατική συνάρτηση της κλάσης Arrays.
Η γενική μορφή της arraycopy έχει ως εξής:

```
public static void arraycopy(Object source, int posAtSource,
Object destination, int posAtDest, int len)
```

Όπως φαίνεται από τη γενική μορφή, η συνάρτηση απαιτεί τις ακόλουθες παραμέτρους:

1. source είναι ο πίνακας του οποίου αντίγραφο θέλουμε να παράγουμε.

2. posAtSource είναι η θέση στον πίνακα source από την οποία επιθυμούμε να αρχίσει η αντιγραφή.
3. destination είναι ο πίνακας-αντίγραφο που θέλουμε να ενημερώσουμε.
4. posAtDest είναι η θέση στον πίνακα destination από όπου επιθυμούμε να αρχίσει η επικόλληση των στοιχείων του source.
5. len είναι το πλήθος των στοιχείων που επιθυμούμε να αντιγράψουμε.

Ο τύπος της πρώτης και της τρίτης παραμέτρου είναι Object. Ο τύπος αυτός παρουσιάζεται εκτεταμένα στην ενότητα 13.1.4. Για την ώρα, αρκεί να γνωρίζουμε ότι κατά την κλήση της arraycopy, μπορούμε να περάσουμε πίνακες στη θέση των παραμέτρων τύπου Object.

```
1   public static void arrayCopyDemo() { //Κώδικας 6.7
2       char[] copyFrom = {'d', 'e', 'c', 'a', 'f', 'f', 'e', 'i', 'n', 'a',
't', 'e', 'd'};
3       char[] copyTo = new char[7];
4        System.arraycopy(copyFrom, 2, copyTo, 0, copyTo.length);
5        System.out.println(new String(copyTo));
    }
```

Κώδικας 6.7 *Παράδειγμα χρήσης της arraycopy*

Στη γραμμή 2 του κώδικα 6.7 ορίζουμε τον πίνακα χαρακτήρων copyFrom. Όπως φανερώνει το όνομά του είναι ο πίνακας που θα αντιγραφεί και πιο συγκεκριμένα μέρος του οποίου θα αντιγράψουμε στον πίνακα copyTo που δηλώνεται και δημιουργείται στη γραμμή 3. Προσέξτε πως το μήκος του copyFrom είναι 13, ενώ του copyTo είναι 7. Η επιλογή αυτή οφείλεται στην πρόθεσή μας να αντιγράψουμε μόνο ένα τμήμα του copyFrom. Αν θέλουμε να αντιγράψουμε ολόκληρο τον πίνακα, τότε ο πίνακας προορισμού πρέπει να δημιουργηθεί σύμφωνα με το μήκος του πίνακα πηγής. Στη γραμμή 4 καλούμε την arraycopy για να πραγματοποιήσουμε την αντιγραφή. Στην κλήση αυτή περνάμε ως πρώτο όρισμα τον copyFrom και ως τρίτο τον copyTo. Το δεύτερο όρισμα σηματοδοτεί τη θέση στον copyFrom στην οποία αρχίζουν τα δεδομένα που θέλουμε να αντιγράψουμε. Εφόσον έχουμε περάσει την τιμή 2, τα δεδομένα θα αρχίσουν να αντιγράφονται από τη δεύτερη θέση του copyFrom, δηλαδή από τον χαρακτήρα ‘c’. Το τελευταίο όρισμα έχει ως αποτέλεσμα το πλήθος των χαρακτήρων που θα αντιγραφεί να είναι 7. Επομένως, στον copyTo θα αντιγραφούν οι χαρακτήρες 'c', 'a', 'f', 'f', 'e', 'i', 'n'. Τέλος, στη γραμμή 5 χρησιμοποιούμε μια βολική μέθοδο για να εκτυπώσουμε τον πίνακα χαρακτήρων copyTo. Αντί να εκτυπώσουμε έναν-έναν τους χαρακτήρες του, προφανώς με έναν κατάλληλο βρόχο, δημιουργούμε ένα String με όρισμα τον copyTo. Όπως ήδη γνωρίζουμε, η print γνωρίζει πώς ακριβώς να εμφανίσει δεδομένα τύπου String.

Σημαντικό είναι να προσέξουμε πως κατά τη χρήση της arraycopy πρέπει να δημιουργήσουμε έναν πίνακα με επαρκές μέγεθος για τα δεδομένα που θα αντιγραφούν. Στην περίπτωση που στη θέση του destination περάσουμε πίνακα μικρότερου μήκους από τα δεδομένα που θα αντιγραφούν θα προκληθεί σφάλμα χρόνου εκτέλεσης.

Ας δούμε τώρα την copyOfRange. Η διεπαφή (interface) της συνάρτησης για πίνακες ακέραιων έχει ως εξής:

```
public static int[] copyOfRange(int[] original, int from,
int to)
```

Το όρισμα int[] σημαίνει πως στη θέση εκείνη μπορούμε να περάσουμε οποιονδήποτε μονοδιάστατο πίνακα ακέραιων. Επίσης, το int[] πριν το όνομα της συνάρτησης σημαίνει πως η συνάρτηση θα επιστρέψει στο περιβάλλον κλήσης έναν μονοδιάστατο πίνακα ακέραιων.

Η συνάρτηση είναι υπερφορτωμένη (overloaded) για όλους τους θεμελιώδεις τύπους. Αυτό σημαίνει πως στη θέση του πίνακα ακέραιων μπορούμε να έχουμε πίνακα οποιουδήποτε θεμελιώδη τύπου. Περισσότερες λεπτομέρειες για την υπερφόρτωση συναρτήσεων στην ενότητα 7.2.